# Terraform Complete Theory Notes

# Introduction to Terraform

Terraform is an Infrastructure as Code (IaC) tool developed by HashiCorp that enables users to provision, manage, and automate infrastructure using code.

Instead of manually creating resources through cloud consoles, Terraform allows infrastructure to be defined and deployed using configuration files.

Terraform supports multiple cloud providers such as:

1. Amazon Web Services
2. Microsoft Azure
3. Google Cloud
4. Kubernetes
5. Docker
6. VMware
7. GitHub

Terraform uses declarative language called HCL (Hashi Corp Configuration Language).

Example:

```
resource "aws_instance" "web" {
  ami           = "ami-123456"
  instance_type = "t2.micro"
}
```

---

# Why Terraform?

## Advantages of Terraform

### 1. Infrastructure as Code

Infrastructure is written in code format.

### 2. Automation

Reduces manual work.

### 3. Reusability

Modules allow reusable infrastructure.

### 4. Version Control

Terraform files can be stored in GitHub.

### 5. Multi-Cloud Support

Supports multiple cloud providers.

### 6. Consistency

Same infrastructure can be recreated anytime.

### 7. Scalability

Easy to scale infrastructure.

### 8. Collaboration

Teams can work together using remote backend.

---

# Terraform Workflow

## Step 1: Write Configuration

Write infrastructure code in `.tf` files.

## Step 2: terraform init

Initializes Terraform.

```
terraform init
```

Downloads:

- Providers
- Plugins
- Backend initialization

## Step 3: terraform validate

Checks syntax.

```
terraform validate
```

### Step 4: terraform plan

Shows execution plan.

```
terraform plan
```

Terraform compares:

- Current infrastructure
- Desired configuration

### Step 5: terraform apply

Creates infrastructure.

```
terraform apply
```

### Step 6: terraform destroy

Deletes infrastructure.

```
terraform destroy
```

---

# Important Terraform Files

### main.tf

Contains main infrastructure code.

### variables.tf

Contains variable declarations.

### terraform.tfvars

Contains variable values.

### outputs.tf

Displays output values.

### provider.tf

Contains provider configuration.

### versions.tf

Specifies Terraform and provider versions.

---

# Terraform Block

```
terraform {
  required_version = ">= 1.0"
}
```

Used to:

- Define Terraform version
- Configure backend
- Define required providers

---

# Provider Block

Provider helps Terraform communicate with cloud platforms.

Example:

```
provider "aws" {
  region = "ap-south-1"
}
```

---

# Resource Block

Resource block creates infrastructure.

Syntax:

```
resource "resource_type" "resource_name" {
}
```

Example:

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}
```

---

# Variables in Terraform

Variables make Terraform reusable.

## Variable Declaration

```
variable "instance_type" {
  type = string
}
```

## Variable Usage

```
instance_type = var.instance_type
```

## Variable Value

```
instance_type = "t2.micro"
```

---

# Types of Variables

## String

```
type = string
```

## Number

```
type = number
```

## Bool

```
type = bool
```

### List

```
type = list(string)
```

### Map

```
type = map(string)
```

---

# Output Block

Used to display values.

Example:

```
output "vpc_id" {
  value = aws_vpc.main.id
}
```

---

# Terraform State File

Terraform stores infrastructure information in:

```
terraform.tfstate
```

State file contains:

- Resource IDs
- Metadata
- Dependency mapping
- Current infrastructure details

---

# Local State vs Remote State

### Local State

Stored locally.

### Remote State

Stored remotely in:

- S3
- Terraform Cloud
- Azure Storage

Example:

```
terraform {
  backend "s3" {
    bucket = "terraform-state-bucket"
    key    = "prod/terraform.tfstate"
    region = "ap-south-1"
  }
}
```

---

## State Locking

Prevents multiple users from modifying infrastructure simultaneously.

Used with:

- DynamoDB
- Terraform Cloud

---

## Terraform Commands

### Initialization

```
terraform init
```

### Formatting

```
terraform fmt
```

### Validation

```
terraform validate
```

### Plan

```
terraform plan
```

### Apply

```
terraform apply
```

### Destroy

```
terraform destroy
```

### Show State

```
terraform show
```

### List Resources

```
terraform state list
```

---

# Dependency in Terraform

Terraform automatically detects dependencies.

Example:

```
resource "aws_subnet" "subnet1" {
  vpc_id = aws_vpc.main.id
}
```

Subnet depends on VPC.

---

# Explicit Dependency

```
depends_on = [aws_internet_gateway.igw]
```

Used when Terraform cannot automatically determine dependency.

---

# Terraform Functions

Functions simplify operations.

## element()

```
element(var.subnets, count.index)
```

## length()

```
length(var.subnets)
```

## upper()

```
upper("terraform")
```

## lower()

```
lower("TERRAFORM")
```

---

# count in Terraform

Creates multiple resources.

Example:

```
resource "aws_instance" "web" {
  count = 2
}
```

---

# for_each in Terraform

Creates resources dynamically.

Example:

```
for_each = toset(["dev", "prod"])
```

---

# Dynamic Blocks

Used for repeated nested blocks.

Example:

```
dynamic "ingress" {
  for_each = var.ports
}
```

---

# Data Sources

Used to fetch existing infrastructure.

Example:

```
data "aws_ami" "ubuntu" {
  most_recent = true
}
```

---

# Terraform Modules

Modules help organize reusable infrastructure.

## Advantages

- Reusability
- Standardization
- Easy maintenance
- Scalability

Example:

```
module "vpc" {
  source = "./modules/vpc"
}
```

---

# Provisioners

Provisioners execute scripts on resources.

## local-exec

Runs command locally.

## remote-exec

Runs command remotely.

Example:

```
provisioner "local-exec" {
  command = "echo Hello"
}
```

---

# Terraform Lifecycle Rules

## create_before_destroy

```
lifecycle {
  create_before_destroy = true
}
```

## prevent_destroy

```
lifecycle {
  prevent_destroy = true
}
```

### ignore_changes

```
lifecycle {
  ignore_changes = [tags]
}
```

---

# Taint and Untaint

### Taint

Marks resource for recreation.

```
terraform taint aws_instance.web
```

### Untaint

```
terraform untaint aws_instance.web
```

---

# Terraform Import

Imports existing infrastructure.

```
terraform import aws_instance.web i-123456
```

---

# Route Tables in Terraform

Route tables define traffic routing.

Example:

```
resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
```

```
  }
}
```

---

# NAT Gateway

NAT Gateway allows private subnet instances to access internet without exposing them publicly.

Important:

- NAT Gateway must be placed in public subnet.
- Requires Elastic IP.

---

# Security Groups

Security Groups act as virtual firewalls.

## Inbound Rules

Control incoming traffic.

## Outbound Rules

Control outgoing traffic.

Example:

```
resource "aws_security_group" "web_sg" {
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

---

# 3-Tier Architecture Using Terraform

## Frontend Layer

Public subnet.

### Backend Layer

Private subnet.

### Database Layer

Private isolated subnet.

Components:

- VPC
- Public Subnets
- Private Subnets
- Internet Gateway
- NAT Gateway
- Route Tables
- Security Groups
- EC2 Instances

---

# Best Practices in Terraform

### Use Variables

Avoid hardcoding.

### Use Modules

Keep code reusable.

### Use Remote Backend

Avoid local state.

### Use State Locking

Avoid concurrent changes.

### Use Naming Standards

Keep resources organized.

### Store Secrets Securely

Avoid hardcoding passwords.

### Use Separate Environments

Examples:

- dev
- stage
- prod

---

# Terraform Folder Structure

```
terraform-project/
│
├── main.tf
├── variables.tf
├── terraform.tfvars
├── outputs.tf
├── provider.tf
├── backend.tf
├── versions.tf
│
├── modules/
│   ├── vpc/
│   ├── ec2/
│   └── security-group/
```

---

# Terraform Interview Questions

### What is Terraform?

Infrastructure as Code tool used to automate infrastructure provisioning.

### Difference between count and for_each?

- count uses index
- for_each uses key-value pair

### What is Terraform state?

Stores infrastructure mapping and metadata.

### What is remote backend?

Stores state remotely.

### What is dependency?

Relationship between resources.

### What is taint?

Forces recreation of resource.

### What is module?

Reusable Terraform configuration.

---

# Key Learnings from My Terraform Journey

- Learned Infrastructure as Code concepts
- Understood VPC, subnetting, route tables, NAT Gateway
- Built 3-tier architecture using Terraform
- Worked with variables and outputs
- Understood Terraform state management
- Learned dependency handling
- Practiced modular infrastructure
- Learned production-level Terraform structure
- Understood Security Groups and routing concepts
- Learned reusable and scalable infrastructure provisioning

---

# Final Thoughts

Terraform is one of the most important DevOps tools for cloud automation. It simplifies infrastructure provisioning, improves consistency, and enables scalable cloud architecture.

Learning Terraform helps in:

- DevOps Engineering
- Cloud Engineering
- Site Reliability Engineering
- Platform Engineering
- Infrastructure Automation

Continuous hands-on practice is the best way to master Terraform.

---

# LinkedIn Post Version

🚀 Recently, I completed hands-on learning and practice in Terraform and explored how Infrastructure as Code (IaC) helps automate cloud infrastructure efficiently.

Here are some of the concepts I learned during my Terraform journey:

✅ Infrastructure as Code (IaC) ✅ Terraform workflow (`init`, `plan`, `apply`, `destroy`) ✅ Providers and Resources ✅ Variables and Outputs ✅ Terraform State Management ✅ Remote Backend and State Locking ✅ Route Tables and Networking Concepts ✅ Internet Gateway and NAT Gateway ✅ Security Groups ✅ count, for_each, and dynamic blocks ✅ Modules and reusable infrastructure ✅ 3-Tier Architecture deployment using Terraform ✅ Production-level Terraform folder structure

🛠️ Hands-on projects completed:

- Created VPCs and subnets
- Configured public and private routing
- Built secure Security Groups
- Provisioned EC2 instances using Terraform
- Implemented a 3-tier AWS architecture

📚 Key Takeaways: Terraform makes infrastructure provisioning:

- Faster
- Reusable
- Scalable
- Consistent
- Automated

This journey helped me strengthen my understanding of Cloud and DevOps practices, especially around AWS infrastructure automation.

Looking forward to learning more about:

- Terraform Modules
- CI/CD Integration
- Kubernetes Automation
- Production Infrastructure Design

#Terraform #DevOps #AWS #InfrastructureAsCode #CloudComputing #Automation #Linux #Cloud #IaC #HashiCorp #TerraformAWS